

Rechnernetze Übungsblatt 6

Finja Kußel 1727369

Aufgabe 1:

Sliding Window:

Mechanismus, der es ermöglicht mehrere Pakete gleichzeitig zu verschicken, bevor eine Bestätigung für das Erste empfangen wurde. Die Fenstergröße bestimmt die Anzahl an Paketen, die maximal unterwegs sein dürfen, für die keine Bestätigung zurück gekommen ist. Sobald ein Paket bestätigt wurde, wird das nächste losgeschickt und das Fenster rutscht weiter.

TCP Tahoe:

Bei diesem Prinzip wird mit einer Fenstergröße von 1 gestartet. Diese lässt man linear wachsen. Wird ein Paketverlust erkannt, entweder durch 3 Duplicate ACKs oder durch einen Timeout wird das Fenster radikal auf 1 zurück gesetzt.

TCP Reno:

Funktioniert wie Tahoe, dass es mit Fenstergröße 1 startet und linear wächst. Es reagiert auch bei 3 Duplicate ACKs und sendet das verlorene Paket erneut, ohne auf ein Timeout zu warten. Anstatt die Fenstergröße radikal zu kürzen, wird es nur halbiert.

TCP Vegas:

Hier wird nicht auf ein Paketverlust gewartet. Vegas misst kontinuierlich die RTT der Segmente, also den Abstand zwischen dem Senden und dem Eintreffen des ACKs. Steigt die Übertragungszeit über die erwartete RTT oder kommt zusätzlich ein Duplicate ACK an, wird reagiert und sendet das betroffene Segment direkt erneut. Aus dem RTT wird zusätzlich eine passende Fenstergröße abgeleitet. Dadurch wird vorgebeugt, dass Pakete verloren gehen.

Protokoll	Schicht
DNS	7. Application Layer
mDNS	7. Application Layer
DHCP	7. Application Layer
BOOTP	7. Application Layer
http(S)	7. Application Layer
POP	7. Application Layer
iMap	7. Application Layer
smtp	7. Application Layer
UDP	4. Transport Layer
TCP	4. Transport Layer
IP	3. Network Layer
ICMP	3. Network Layer

ARP	3. Network Layer
RARP	3. Network Layer
MAC	2. Link Layer
PUP	3. Network Layer
RIP	3. Network Layer
RIPNG	3. Network Layer
BGP	3. Network Layer
NTP	7. Application Layer

Aufgabe 2

- Wie viele Hosts befinden sich in ihrem lokalen Klasse-C-Netz? – 7
`nmap -sn 10.42.1.0/24`
- Welches Betriebssystem wird von scanme.nmap.org verwendet? – zu 96% Linux (Kernel 5.0-5.14)
`sudo nmap -O scanme.nmap.org`
- An welchem Datum wurde die Webseite nmap.org registriert? – 18.01.1999
`whois nmap.org`
- Wie kann man möglichst effektiv eine größere Menge an Adressen nach offenen TCP-Ports scannen?
`sudo nmap -T4 --min-rate 1000 -p 1-1000 10.42.1.0/24`
- Wie funktioniert der SYN-Scan und für was kann man ihn verwenden?
`sudo nmap -sS 10.42.1.0/24`
 sendet nur ein SYN, wartet auf SYN-ACK, schickt dann RST statt den Handshake zu vervollständigen Vorteil: schneller, unauffälliger, keine vollständige Verbindung im Log der Anwendung
- Welches sind die offenen Ports, die bei Ihren bisherigen Nmap-Scans am häufigsten auftreten, und wofür werden sie verwendet?
 Ports 22 (SSH) verwendet für verschlüsselte Fernzugriffe auf einen Rechner und Ports 80 (http) verwendet von Webservern

Aufgabe 3

- Laptop verbindet sich neu im WLAN und hat keine IP, braucht aber automatisch eine gültige IP-Adresse, Gateway, DNS-Server, ohne dass man das manuell konfiguriert
- Filter: `udp port 67 or udp port 68`
- Discover** (Client → Broadcast): Client hat noch keine IP, fragt "ist irgendein DHCP-Server da?" per Broadcast (Ziel-IP 255.255.255.255), da er noch keinen Server kennt.
Offer (Server → Client): Der DHCP-Server antwortet mit einem Vorschlag für eine freie IP-Adresse plus Zusatzinfos (Lease Time, Subnetzmaske, Gateway).

Request (Client → Broadcast): Der Client "bestätigt" öffentlich per Broadcast, dass er genau dieses Angebot annehmen möchte – wichtig, falls mehrere DHCP-Server im Netz geantwortet haben, damit die anderen wissen, dass ihr Angebot nicht gewählt wurde.

Ack (Server → Client): Der Server bestätigt final die Zuteilung, der Client darf die IP jetzt tatsächlich nutzen.

```
> User Datagram Protocol, Src Port: 68, Dst Port: 67
< Dynamic Host Configuration Protocol (Discover)
  Message type: Boot Request (1)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0x91b28d0f
  Seconds elapsed: 0
  > Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 0.0.0.0
  Next server IP address: 0.0.0.0
  Relay agent IP address: 0.0.0.0
  Client MAC address: d2:f0:e2:79:96:64 (d2:f0:e2:79:96:64)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  > Option: (53) DHCP Message Type (Discover)
  > Option: (55) Parameter Request List
  > Option: (57) Maximum DHCP Message Size

> User Datagram Protocol, Src Port: 68, Dst Port: 67
< Dynamic Host Configuration Protocol (Request)
  Message type: Boot Request (1)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0x2405463e
  Seconds elapsed: 0
  > Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 0.0.0.0
  Next server IP address: 0.0.0.0
  Relay agent IP address: 0.0.0.0
  Client MAC address: 32:d9:92:ae:6c:a2 (32:d9:92:ae:6c:a2)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  > Option: (53) DHCP Message Type (Request)
  > Option: (61) Client identifier
  > Option: (50) Requested IP Address (10.42.1.194)
```

```

> User Datagram Protocol, Src Port: 67, Dst Port: 68
> Dynamic Host Configuration Protocol (Offer)
  Message type: Boot Reply (2)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0x91b28d0f
  Seconds elapsed: 0
> Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 10.42.1.109
  Next server IP address: 10.42.1.1
  Relay agent IP address: 0.0.0.0
  Client MAC address: d2:f0:e2:79:96:64 (d2:f0:e2:79:96:64)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
> Option: (53) DHCP Message Type (Offer)
> Option: (54) DHCP Server Identifier (10.42.1.1)
> Option: (51) IP Address Lease Time

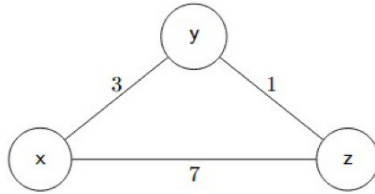
> User Datagram Protocol, Src Port: 67, Dst Port: 68
> Dynamic Host Configuration Protocol (ACK)
  Message type: Boot Reply (2)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0x91b28d0f
  Seconds elapsed: 0
> Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 10.42.1.109
  Next server IP address: 10.42.1.1
  Relay agent IP address: 0.0.0.0
  Client MAC address: d2:f0:e2:79:96:64 (d2:f0:e2:79:96:64)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
> Option: (53) DHCP Message Type (ACK)
> Option: (54) DHCP Server Identifier (10.42.1.1)
> Option: (51) IP Address Lease Time

```

Aufgabe 4

- (a) Jeder Router kennt anfangs nur die Kosten zu seinen direkten Nachbarn. In jeder Runde tauschen benachbarte Router ihre komplette Distanztabelle aus. Jeder Router berechnet dann für jedes Ziel:
- neueDistanz(zu Ziel Z) = min über alle Nachbarn N von:
- Kosten(ich -> N) + Distanz(N-> Z), die N mir gemeldet hat
- Das wird iterativ wiederholt, bis sich nichts mehr ändert.
- Beispiel: A kennt nur B (3) und C (23). Nachdem B seine Tabelle an A meldet (B->C=2), rechnet A: A->C via B=3+2=5, was besser als 23 ist. A aktualisiert nach C usw.

b) Bestimmen Sie die Routingtabellen in jedem Schritt, indem Sie den Distanz-Vektor-basierten Algorithmus anwenden. Geben Sie danach den kostengünstigsten Weg von Router „z“ zu Router „x“ an. Sie können die Lösungen einfach direkt in die Tabellen der PDF einfügen und diese in den Pull Request hinzufügen.



Von x	Via x	Via y	Via z
Zu x	0		
Zu y		3	
Zu z			7

Von y	Via x	Via y	Via z
Zu x	3		
Zu y		0	
Zu z			1

Von z	Via x	Via y	Via z
Zu x	7		
Zu y		1	
Zu z			0

Von x	Via x	Via y	Via z
Zu x			
Zu y			8
Zu z		4	

Von y	Via x	Via y	Via z
Zu x			8
Zu y			
Zu z	10		

Von z	Via x	Via y	Via z
Zu x		4	
Zu y	10		
Zu z			

Von x	Via x	Via y	Via z
Zu x	0		
Zu y		3	
Zu z		4	

Von y	Via x	Via y	Via z
Zu x	3		
Zu y		0	
Zu z			1

Von z	Via x	Via y	Via z
Zu x		4	
Zu y		1	
Zu z			0

c) Die Kosten zwischen „x“ und „y“ steigen nun von 3 auf 7. Berechnen Sie die neuen Routingtabellen. Ändert sich der kostengünstigste Pfad von „z“ nach „x“?

Von x	Via x	Via y	Via z
Zu x	0		
Zu y		7	
Zu z			7

Von y	Via x	Via y	Via z
Zu x	7		
Zu y		0	
Zu z			1

Von z	Via x	Via y	Via z
Zu x	7		
Zu y		1	
Zu z			0

Von x	Via x	Via y	Via z
Zu x	0		
Zu y		7	
Zu z			7

Von y	Via x	Via y	Via z
Zu x	7		
Zu y		0	
Zu z			1

Von z	Via x	Via y	Via z
Zu x	7		
Zu y		1	
Zu z			0

Von x	Via x	Via y	Via z
Zu x			
Zu y			
Zu z			

Von y	Via x	Via y	Via z
Zu x			
Zu y			
Zu z			

Von z	Via x	Via y	Via z
Zu x			
Zu y			
Zu z			

d) Setzen Sie sich den weiteren Gedanken an Router „B“ und „C“ auf einen anderen. Verarbeiten Sie die

(d) **Distanzvektor (Bellman-Ford):** B und E merken den Ausfall von D sofort, da ihre direkte Verbindung wegfällt. A und C erfahren es aber erst nach mehreren Austauschrunden. Dabei kann es einem Problem kommen: B und E melden sich gegenseitig fälschlicherweise als Alternativroute zu D, weil sie die veraltete Info des anderen übernehmen. Dies löst sich erst nach mehreren Runden auf.

Link-State: B und E bemerken den Ausfall und schicken direkt eine LSA an alle Router. Jeder berechnet mit Dijkstra selbst neu. Daher wissen A und C schon nach einer Runde Bescheid, deutlich schneller und ohne Risiko für ein Problem.

Fazit: Link-State konvergiert schneller und zuverlässiger, braucht aber mehr Rechenleistung und Bandbreite